Name: Chaitra Samant
Branch: TY Computer Engg
Reg No: 231070055
Batch: B

EXPERIMENT 08 - MIS LAB

AIM: To prepare a Software Quality Assurance (SQA) Plan for the Customer Complaint Intelligence System (CCIS) — an AI-driven Management Information System for Corporate Decision Making.

SOFTWARE QUALITY ASSURANCE PLAN
Customer Complaint Intelligence System for Corporate Decision Making

THEORY

1. What is Software Quality?

Software quality is the degree to which a software product conforms to explicitly stated functional and non-functional requirements, adheres to documented development standards, and meets the implicit expectations of all stakeholders. It encompasses two complementary perspectives:

- Functional Quality: The system correctly performs its intended functions — in the case of CCIS, this means accurate complaint ingestion, reliable AI/NLP classification, correct sentiment scoring, and precise churn-risk prediction.
- Structural Quality: The internal properties of the codebase — including maintainability, testability, security, performance,

and reliability — that determine how well the system is built, not just what it does.

The ISO/IEC 25010 quality model organises software quality into eight characteristics: Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, and Portability. For CCIS — which handles sensitive corporate complaint data and exposes AI-generated insights to senior executives — all eight characteristics are critical, with Security and Reliability ranked highest.

2. What is SQA (Software Quality Assurance)?

Software Quality Assurance (SQA) is a systematic, process-oriented discipline that defines and enforces the activities, standards, procedures, and tools used to ensure that a software product meets its quality objectives throughout the entire Software Development Life Cycle (SDLC). SQA is proactive by design: rather than simply finding defects after they appear, it builds quality into every phase of development.

Key distinctions:

- SQA vs. Testing: Testing is a subset of SQA. While testing detects defects in the product, SQA encompasses the broader process — standards definition, reviews, audits, process monitoring, and continuous improvement.
- SQA vs. QC (Quality Control): Quality Control is product-focused (inspect the output); SQA is process-focused (improve the process that creates the output).

For the CCIS, SQA is especially important because the system relies on probabilistic AI/NLP models whose outputs cannot be verified

by simple pass/fail unit tests. SQA provides the governance framework — human-in-the-loop reviews, model validation criteria, and bias audits — that ensures AI-generated complaint classifications and churn-risk scores are trustworthy enough to drive corporate decisions.

3. Components of an SQA Plan

An SQA Plan is the master document that formalises the quality strategy for a project. It typically comprises the following components:

Component	Purpose
Project Overview	Identifies the system, its purpose, and the scope of the SQA plan.
Quality Objectives	Defines measurable quality goals aligned to project requirements.
SQA Activities	Lists the concrete quality assurance activities (reviews, audits, walkthroughs).
Testing Strategy	Specifies the types, levels, tools, and entry/exit criteria for testing.
Roles & Responsibilities	Assigns SQA duties to specific team members.
Standards & Procedures	Defines coding standards, documentation standards, and review procedures.
Defect Management	Describes the defect lifecycle, classification, tracking, and resolution process.
Tools & Techniques	Lists the software tools and quality techniques to be applied.

TASKS

The following tasks constitute the complete SQA Plan for the Customer Complaint Intelligence System (CCIS) for Corporate Decision Making

Task 1: Project Overview

1.1 System Identification

System Name: Customer Complaint Intelligence System (CCIS)

System Type: AI-Driven Management Information System (MIS)

1.2 System Description

The CCIS is a multi-module, intelligence-driven MIS that consolidates customer complaints from corporate channels — CRM systems, call centres, email, and web portals — and transforms raw grievance data into actionable decision intelligence. The system architecture (established in Lab 3) comprises:

- Frontend Portal: Next.js 14 + Tailwind CSS — complaint submission and executive dashboards.
- Backend & Database: Supabase (managed PostgreSQL) with Row-Level Security and REST APIs.
- AI/NLP Engine: Python + LangChain + Hugging Face models for complaint classification, sentiment analysis, root-cause detection, and churn-risk prediction.
- CI/CD: GitHub Actions + Vercel deployment pipeline.

1.3 SQA Plan Scope

This SQA Plan applies to all phases of the CCIS SDLC, from requirements validation through to post-deployment maintenance — as defined in the 63-day project schedule (Lab 6). It covers all 12 WBS tasks (T1–T12) and all 10 deliverables (D1–D10) identified in Lab 7.

1.4 Reference to Previous Labs

Lab	Experiment	Contribution to SQA Plan
Lab 1	Problem Statement	Defines the quality problem domain: complaint misclassification, data silos, management blindspots.
Lab 2	Software Scoping	Establishes in-scope and out-of-scope boundaries that define the SQA coverage perimeter.
Lab 3	Resource Estimation	Identifies the technology stack subject to quality standards and testing.
Lab 4	Cost & Effort Estimation	Provides COCOMO-based effort baseline for allocating QA effort (typically 15–20% of total effort).
Lab 5	Risk Analysis	Risk register (R01–R08) directly informs SQA priorities and defect severity classification.
Lab 6	Project Schedule	WBS tasks T9–T11 (Testing, UAT, Bug Fixing) are the SQA execution timeline backbone.
Lab 7	Project Plan	Roles, deliverables, and tools tables provide the SQA resource and output framework.

Task 2: Quality Objectives

The quality objectives for CCIS are derived from the ISO/IEC 25010 quality model, the non-functional requirements defined in Lab 2, and the risk register from Lab 5. Each objective is measurable and assigned an acceptance threshold.

#	Quality Attribute	Objective	Metric	Target
1	Functional Correctness	AI classifier assigns the correct complaint category.	Classification Accuracy	≥ 90%
2	Functional Correctness	Sentiment analysis correctly identifies positive/negative/neutral tone.	Sentiment F1-Score	≥ 0.85
3	Performance Efficiency	Executive dashboard refreshes within acceptable time after data update.	Page Load / Refresh Time	≤ 30 seconds
4	Reliability	System available during business hours without unplanned downtime.	Uptime	≥ 99%
5	Security	No critical or high-severity security vulnerabilities at deployment.	Penetration Test Result	Zero Critical/High findings

6	Maintainability	Code is clean, documented, and passes static analysis.	Code Coverage	≥ 80%
7	Usability	End users can complete complaint submission without assistance.	Task Success Rate (UAT)	≥ 95%
8	Data Quality	Complaints ingested with complete mandatory fields.	Data Completeness Rate	≥ 98%
9	Defect Density	Low number of defects per KLOC in delivered code.	Defects per KLOC	< 5 defects/KLOC
10	Scalability	System handles growing complaint volumes without degradation.	Throughput under Load	500 concurrent users

Note: Quality Objective 1 (AI Classification Accuracy ≥ 90%) directly mitigates Risk R02 (AI/ML Accuracy Risk) identified in Lab 5. Quality Objectives 5 and 8 directly address Risks R05 (Security) and R01 (Data Quality) respectively.

Task 3: SQA Activities

SQA activities are embedded into every phase of the 63-day CCIS project schedule. The following activities are planned across the five project phases (Planning, Design, Development, Testing, Closure).

3.1 Requirements Review

Phase: Planning (T1–T2, Days 1–10)

- Conduct a formal Requirements Walkthrough with all stakeholders — Project Manager, Full-Stack Developer, AI/NLP Engineer, Data Analyst, and QA Engineer — to verify that the requirements documented in Lab 2 are complete, unambiguous, testable, and traceable.
- Verify that all in-scope functional requirements (FR1–FR10) and non-functional requirements (NFR1–NFR5) from Lab 2 have corresponding acceptance criteria.
- Produce a Requirements Traceability Matrix (RTM) linking each requirement to its design artefact, implementation module, and test case.

3.2 Design Review

Phase: Design (T3–T4, Days 11–22)

- Peer review of the system architecture diagram (Next.js frontend, Supabase backend, Python AI engine) for completeness, security design, and alignment with the technology stack defined in Lab 3.
- Review the Supabase database schema design for normalisation, referential integrity, and Row-Level Security policy correctness.

- Verify that the LangChain pipeline design includes error handling, fallback mechanisms, and human-in-the-loop hooks (as required by R02 mitigation in Lab 5).

3.3 Code Reviews

Phase: Development (T5–T8, Days 23–52)

- Enforce mandatory pull request (PR) reviews on GitHub for every feature branch merge. Minimum: one peer reviewer approval before merge to main.
- Apply coding standards (Section 6) to all Next.js, Python, and SQL code. Automated linting (ESLint, Flake8) must pass before PR approval.
- Conduct focused security code reviews for all authentication routes, RBAC middleware, and any module that processes PII (Personally Identifiable Information) — directly addressing Risk R05.

3.4 AI Model Validation Reviews

Phase: Development (T6, Days 23–36)

- Before integrating the NLP classification model into the production pipeline, conduct a model validation review: evaluate precision, recall, and F1-score on a held-out test dataset of at least 200 labelled complaints.
- Document model card: training data source (CFPB/Kaggle as noted in Lab 3), model version, known limitations, and bias assessment.
- Establish a human-in-the-loop review queue for complaints where model confidence score is below 0.75.

3.5 Process Audits

Phase: Throughout (Weekly)

- Conduct a weekly SQA audit (15-minute sprint retrospective checkpoint) to verify that development processes — branching strategy, testing before merge, documentation updates — are being followed.
- Audit CI/CD pipeline (GitHub Actions) to confirm all automated tests pass on every commit.

3.6 User Acceptance Testing (UAT) Support

Phase: Testing (T10, Days 53–59)

- Coordinate UAT sessions with representative users: Operations Managers, Senior Management / Executives, and Customer Service Supervisors (user roles defined in Lab 2).
- Prepare UAT test scripts and scenario packs covering all primary user journeys: complaint submission, dashboard review, report generation, and alert acknowledgement.
- Collect and formally log all UAT feedback in the defect tracking system (Section 7).

Task 4: Testing Strategy

4.1 Overview

The CCIS testing strategy follows a layered approach: testing is applied at every level of the system — from individual functions to end-to-end user scenarios — and integrated into the CI/CD pipeline so quality is continuously verified throughout development.

4.2 Testing Levels

Level	Scope	Description	Tool
Unit Testing	Individual functions & components	Tests for React components (Jest), Python API handlers (PyTest), and LangChain pipeline functions. Each module tested in isolation with mock data.	Jest, PyTest
Integration Testing	Module interfaces	Tests the interaction between frontend (Next.js) and backend (Supabase APIs), and between the Python AI pipeline and the database. Validates data flow end-to-end.	PyTest, Supertest
AI/NLP Model Testing	Classificatio n & Sentiment Engine	Evaluates model accuracy (precision, recall, F1) on labelled complaint datasets. Validates that the 90% accuracy objective is met before production integration.	Python scripts, Scikit-learn metrics

System Testing	Complete integrated system	End-to-end validation of all system flows: complaint ingestion → AI processing → dashboard display. Tests functional requirements FR1–FR10 from Lab 2.	Manual + Selenium/Playwright
Security Testing	Authentication, RBAC, data handling	Penetration testing of login routes, session management, RBAC enforcement, and PII handling. Directly addresses Risk R05 from Lab 5.	OWASP ZAP, manual review
Performance Testing	Dashboard & API response times	Load tests to verify dashboard refresh ≤ 30 seconds and APIs respond acceptably under 500 concurrent users. Validates NFR1 (Performance) from Lab 2.	Locust (Python)
User Acceptance Testing (UAT)	User journeys	Scenario-based testing conducted with representative users (Operations Managers,	Manual, feedback forms

		Executives) to confirm usability and functional fitness. Validates UAT objective ≥ 95%.	
Regression Testing	Previously tested features	Re-run unit and integration test suites after every bug fix (T11) to ensure no regressions are introduced. Automated via GitHub Actions.	Jest, PyTest (CI/CD)

4.3 Test Environment

- Development Environment: Local machines with Node.js, Python 3.10+, Supabase CLI.
- Staging Environment: Vercel Preview Deployment + Supabase staging project (mirrors production).
- Production Environment: Vercel Hobby deployment + Supabase production project.
- AI Model Environment: Hugging Face Inference API (free tier) for model serving.

4.4 Entry and Exit Criteria

Testing Level	Entry Criteria	Exit Criteria
Unit Testing	Feature development complete; code merged to feature branch.	All unit tests pass; code coverage ≥ 80%.
Integration Testing	All unit tests pass; modules deployed to staging.	All integration tests pass; no critical API failures.
System Testing	Integration tests complete; full system deployed to staging.	All FR test cases pass; no open Critical or High defects.
UAT	System testing sign-off obtained; UAT environment ready.	≥ 95% task success rate; all blocking issues resolved.
Security Testing	System deployed to staging; RBAC and auth in place.	Zero Critical/High vulnerabilities; Medium findings documented.
Performance Testing	System deployed to staging; representative data loaded.	Dashboard loads in ≤ 30s; APIs respond under load.

Task 5: Roles and Responsibilities

The CCIS team structure was established in Lab 7. For SQA, specific quality responsibilities are assigned to each role, consistent with the team defined in the project plan.

Role	Count	SQA Responsibilities
Project Manager	1	Owns the SQA Plan; ensures SQA activities are resourced and executed on schedule; signs off on quality gate passage; escalates unresolved Critical defects; conducts weekly process audits.
QA Engineer	1	Primary SQA executor: writes and executes all test cases (unit, integration, system, UAT); manages the defect tracker; produces test reports D7 (Lab 7); coordinates security testing; enforces entry/exit criteria for each testing phase.
Full-Stack Developer	1	Writes unit tests for all Next.js/React components; performs peer code reviews; remediates defects assigned by QA Engineer; ensures ESLint standards are met; participates in requirements and design walkthroughs.
AI/NLP Engineer	1	Owns AI model validation: evaluates classification and sentiment models against quality objectives; maintains the model card; implements human-in-the-loop review queue; remediates AI accuracy defects.
Data Analyst	1	Validates dashboard KPI calculations for correctness; verifies data completeness rates from ingestion pipeline; reviews report outputs for business accuracy; participates in UAT as a domain expert.

Infrastructure Engineer	1	Maintains CI/CD pipeline quality gates (GitHub Actions); enforces automated test runs on every commit; manages staging and production environments; executes performance and security testing tooling.
Stakeholders / End Users	3+	Participate in UAT (Operations Managers, Senior Executives, Customer Service Supervisors); provide UAT sign-off; raise defect reports for business logic failures.

5.1 RACI Matrix — Key SQA Activities

Activity	PM	QA Eng	Dev	AI Eng	DA	Infra
Requirements Review	A	R	C	C	C	I
Design Review	A	R	R	R	C	C
Code Reviews	A	C	R	R	I	I
AI Model Validation	A	C	I	R	C	I
Unit & Integration Testing	A	R	C	C	I	I
Security Testing	A	R	C	I	I	R
UAT Coordination	A	R	I	I	C	I

Defect Management	A	R	C	C	C	C

R = Responsible | A = Accountable | C = Consulted | I = Informed

Task 6: Standards and Procedures

6.1 Coding Standards

All code written for the CCIS must adhere to the following language-specific standards:

- JavaScript / Next.js / React: ESLint with the Airbnb configuration. Prettier for consistent code formatting. React components must be functional (no class components). All components documented with JSDoc comments.
- Python (Backend & AI Pipeline): PEP 8 style guide enforced via Flake8. Black formatter for automatic code formatting. All functions must include docstrings. Type hints required on all function signatures.
- SQL / Supabase Schemas: Lowercase snake_case for all table and column names. All tables must have a primary key and created_at / updated_at timestamps. Row-Level Security (RLS) policies must be explicitly defined for every table.
- General Rules: No hardcoded credentials or API keys in source code; use environment variables. Maximum function length: 50 lines. Maximum file length: 300 lines. All code changes must be made via feature branches — no direct commits to main.

6.2 Documentation Standards

- All public API endpoints must be documented in an OpenAPI 3.0 specification.
- All LangChain prompts and model configurations must be documented in the AI Model Card (per Lab 5 R02 mitigation).
- All database schema changes must be accompanied by a migration file and a description in the change log.
- Test cases must follow the format: Test ID, Description, Pre-conditions, Steps, Expected Result, Actual Result, Pass/Fail.

6.3 Review Procedures

- Requirements Review: Conducted as a structured walkthrough session. All team members attend. Findings logged in GitHub Issues. Review sign-off required before design phase begins.
- Code Review: Initiated via GitHub Pull Request. Reviewer must check against coding standards, test coverage, and security considerations. PR approved only when all automated checks pass and reviewer approves.
- Test Review: QA Engineer reviews all test cases with Project Manager before execution. Test results reviewed at weekly audit checkpoint.

6.4 Change Management Procedure

1. A Change Request (CR) is raised in GitHub Issues with label change-request.
2. Project Manager assesses impact on scope, schedule, cost, and quality.
3. If approved: affected components are re-reviewed, re-tested, and re-documented.

4. If rejected: CR is closed with justification logged.
5. All approved CRs are reflected in the RTM update.

Task 7: Defect Management

7.1 Defect Lifecycle

All defects detected during any SQA activity — code review, testing, UAT, or audit — follow a standardised lifecycle managed in GitHub Issues.

State	Description
New	Defect discovered and logged with full details by tester or reviewer.
Assigned	Project Manager or QA Engineer assigns the defect to the responsible developer.
In Progress	Developer is actively working on the fix.
Fixed	Developer marks as fixed and submits PR with the fix.
In Review	QA Engineer verifies the fix in the staging environment.
Closed	Fix verified; defect closed. RTM updated.
Rejected	Defect deemed not reproducible, not a defect, or out-of-scope.
Deferred	Defect acknowledged but deferred to a future release (Low severity only).

7.2 Defect Classification and Priority

Severity	Priority	Definition (CCIS Context)	Resolution SLA
Critical	P1	System crash, data loss, security breach, complete failure of core module (e.g. complaint ingestion pipeline down, RBAC bypass).	Fix within 24 hours
High	P2	Major functional failure — AI classifier not working, dashboard not loading, authentication failures. Directly tied to R01, R02, R05 risks (Lab 5).	Fix within 48 hours
Medium	P3	Partial functional failure — incorrect chart data, minor UI misalignment, slow API response above threshold. System still usable.	Fix within 1 week
Low	P4	Cosmetic issues — typos, UI spacing, minor style inconsistencies. No functional impact.	Fix before final release or defer

7.3 Defect Logging Template

Every defect logged in GitHub Issues must include the following fields:

- Defect ID: Auto-generated by GitHub Issues.
- Title: Short descriptive title.
- Module: Affected system module (e.g. Complaint Intake Portal, AI/NLP Engine, Dashboard).
- Severity & Priority: As per classification table above.
- Environment: Where found (development / staging / UAT / production).
- Steps to Reproduce: Numbered, step-by-step reproduction instructions.
- Expected Result: What should happen.
- Actual Result: What actually happened.
- Attachments: Screenshots, log files, or model output samples.
- Detected By: Tester's name and date.
- Assigned To: Developer responsible for fix.

7.4 Defect Metrics

The QA Engineer will track and report the following defect metrics weekly:

- Defect Discovery Rate: Number of new defects found per testing phase.
- Defect Resolution Rate: Number of defects closed per week.
- Defect Density: Total defects per KLOC (target: < 5 defects/KLOC).
- Defect Leakage: Number of defects found in UAT or production that escaped earlier testing phases.

- Reopen Rate: Percentage of closed defects reopened due to inadequate fixes.

Task 8: Tools and Techniques

The following tools and quality techniques are employed across all CCIS SQA activities. The tools are aligned with the technology stack established in Labs 3 and 7.

8.1 Testing Tools

Tool	Category	Usage in CCIS
Jest	Unit & Integration Testing	Tests all Next.js/React components and frontend API calls. Generates code coverage reports.
PyTest	Unit & Integration Testing	Tests all Python backend functions, FastAPI/Flask routes, and LangChain pipeline components.
Supertest / Playwright	System & E2E Testing	End-to-end testing of complete user workflows: complaint submission, AI processing, dashboard review.
Locust	Performance Testing	Simulates concurrent users to validate dashboard refresh time ≤ 30s and API throughput under load.
OWASP ZAP	Security Testing	Automated security scanning of web application endpoints. Identifies OWASP Top 10 vulnerabilities.
Scikit-learn Metrics	AI Model Testing	Evaluates NLP model precision, recall, F1-score, and confusion matrix against labelled test datasets.

8.2 Code Quality Tools

Tool	Category	Usage in CCIS
ESLint (Airbnb Config)	Static Analysis	Enforces JavaScript/React coding standards on every commit via pre-commit hooks.
Flake8 + Black	Static Analysis / Formatting	Enforces PEP 8 compliance and consistent formatting for all Python code.
Istanbul (NYC)	Code Coverage	Measures unit test coverage for JavaScript; integrated with Jest. Target ≥ 80%.
GitHub Actions	CI/CD Quality Gates	Runs ESLint, Flake8, Jest, and PyTest automatically on every pull request before merge is allowed.

8.3 Project Management & Defect Tracking Tools

Tool	Category	Usage in CCIS
GitHub Issues & Milestones	Defect Tracking / Sprint Manageme nt	Central defect tracker and sprint board. Labels used for severity (critical, high, medium, low) and type (bug, enhancement, change-request).
Figma	UI/UX Review	Dashboard prototypes reviewed for usability before frontend development. Reduces UI-related defects in later phases.

Google Sheets / Excel	Test Case Management	Test case repository: stores all test cases with ID, description, steps, expected result, status.
Notion / Confluence	Documentation	Stores SQA Plan, Requirements Traceability Matrix, review minutes, and model cards.

8.4 SQA Techniques Summary

Technique	Application in CCIS
Requirements Walkthrough	Structured group review of requirements to detect ambiguities and gaps before design begins.
Peer Code Review	Mandatory GitHub PR review enforcing coding standards, security, and test coverage.
Test-Driven Development (TDD)	Write unit tests before implementing Python backend functions for AI pipeline components.
Boundary Value Analysis	Test complaint ingestion with boundary inputs: empty submissions, maximum-length narratives, edge-case dates.
Equivalence Partitioning	Partition complaint categories and sentiment classes to design efficient test cases covering all valid/invalid inputs.
Risk-Based Testing	Prioritize testing of modules associated with Critical/High risks from Lab 5 (data

	ingestion for R01, AI engine for R02, auth module for R05).
Regression Testing	Re-run full test suite after every bug-fix sprint (T11) to prevent re-introduction of previously fixed defects.
Model Validation (AI-specific)	Evaluate NLP model on held-out labelled dataset; compute confusion matrix; compare against quality objective thresholds.

CONCLUSION

Hence, in this experiment, the development of a comprehensive **Software Quality Assurance (QA) Plan** was achieved by synthesizing four critical pillars of quality governance: **Standardization, Procedural Testing, AI-Specific Validation, and Accountability.**

By aligning ten measurable objectives with the ISO/IEC 25010 standard, the plan ensures that the CCIS meets global benchmarks for reliability and security. The procedural framework integrates a multi-level testing strategy—from unit to user acceptance—directly into the 63-day project timeline, governed by strict entry and exit criteria. Crucially, the plan addresses the probabilistic nature of AI by introducing specialized validation techniques, such as held-out dataset evaluations and human-in-the-loop governance, to ensure the accuracy of sentiment analysis and NLP classification. Finally, through a RACI-based roles framework and a rigorous defect lifecycle, the experiment demonstrates that a structured SQA plan transforms quality from an afterthought into a continuous, actionable process that guarantees trustworthy, data-driven insights.